AuthShield - How Your Backend Works

A step-by-step teaching book built from your own code

Written for Rishikesh

October 2026

- How to use this book
  - Running AuthShield on your machine
- Phase 1: Setting up the project
  - Step 1. Create the FastAPI project
  - Step 2. Create API versioning and routing
- Phase 2: Building the security foundation
  - Step 3. Create the security module
  - Step 4. Add environment variables and JWT configuration
  - Step 5. Implement access-token creation
  - Step 6. Implement access-token decoding
- Phase 3: Creating users and their storage
  - Step 7. Create the user repository
- Phase 4: Registration and login
  - Step 8. Create the authentication service
  - Step 9. Create the authentication schemas
  - Step 10. Create the authentication routes
- Phase 5: Protecting endpoints
  - Step 11. Create the dependency module
  - Step 12. Create the user routes
- Phase 6: Managing sessions and logout
  - Step 13. Create the session repository
  - Step 14. Create the token-revocation repository
  - Step 15. Implement logout
- Phase 7: Refresh tokens and rotation
  - Step 16. Add refresh-token creation and decoding
  - Step 17. Implement the refresh endpoint
  - Step 18. Implement refresh-token rotation
- Phase 8: Email verification
  - Step 19. Add email-verification status to users
  - Step 20. Add email-verification token functions
  - Step 21. Connect email verification to registration
- Phase 9: Password reset
  - Step 22. Add password-reset token functions
  - Step 23. Add password-reset business logic
  - Step 24. Add password-reset request schemas
  - Step 25. Add password-reset routes
- Phase 10: How all the files work together
- Phase 11: Tracing a complete login request
- Phase 12: Your complete learning sequence
- Phase 13: What you learn next
- Part II: Verified status and the Stage 1 fixes
  - What was actually tested
  - Stage 1, Fix 1: single-use reset tokens
  - Stage 1, Fix 2: revoke the user’s sessions after a reset
  - The discovery testing forced on us
  - Appendix A: References for going deeper
  - Appendix B: Glossary
  - Appendix C: Postman recipes

How to use this book

This book teaches you backend development through your own project, **AuthShield**. It is organized around your own learning sequence - Phases 1 to 13, Steps 1 to 25 - as a clear teaching order. That order is a reconstruction for learning, not a guaranteed record of the exact historical order in which you wrote each file. Each step adds a small piece of code, and those pieces accumulate into the whole system.

Code samples for the existing project are your real code, copied from the ZIP you sent me. Where your earlier lesson notes showed code only “conceptually”, this book fills in the actual code and says so. Where the book adds new code - the Stage 1 fixes in Part II - it is clearly marked as new, tested work that you have not applied yet.

Each step follows the same pattern:

1. **What we did** - the piece we built, in one or two sentences.
2. **The code** - your actual code.
3. **Line by line** - what each line means and why we wrote it at that point.
4. **What we learned** - the concepts this step teaches.
5. **Check it yourself** - where useful, how to prove the behavior in Postman.

Three honest ground rules:

- Chapters for Phases 1 to 12 explain code that exists today. Phase 13 and the final part of the book show tested improvements you have not applied yet, and they are clearly marked as new work.
- Claims about behavior were verified by actually running your app, not only by reading it. The chapter “What was actually tested” lists every test and its result.
- Your project is a learning prototype: data lives in memory and resets on restart. That was a deliberate choice, and this book explains when and how to outgrow it.

Running AuthShield on your machine

To follow along with the code and the Postman checks, here is the complete setup from scratch. On Windows, run these in your project folder (`AuthShield/Backend`):

```
python -m venv venv
venv\Scripts\activate
pip install fastapi "uvicorn[standard]" "pwdlib[argon2]" pyjwt python-dotenv email-validator
uvicorn app.main:app --reload
```

What each line does: the first creates an isolated Python environment (a `venv`) so this project’s libraries do not mix with others; the second activates it; the third installs the exact libraries this book’s tests ran with (`pwdlib[argon2]` includes the argon2 password hasher - plain `pwdlib` alone cannot hash passwords); the last starts the server with auto-reload, so saving a file restarts it.

Your `.env` file needs one line, with your own secret value:

```
SECRET_KEY=<your-own-long-random-string>
```

Generate a fresh one for yourself (for example with `python -c "import secrets; print(secrets.token_hex(32))"`) and never commit `.env` or share it. The secret in any shared copy of a project should be changed, because anyone who has it can sign valid tokens.

The server then runs at `http://127.0.0.1:8000` , with the interactive docs at `http://127.0.0.1:8000/docs` .

The tests in this book were run against a copy of your project with Python 3.10.12, FastAPI 0.162.0, PyJWT 2.3.0, pwdlib 0.3.1, python-dotenv 1.2.4, and email-validator 2.3.0.

Phase 1: Setting up the project

Step 1. Create the FastAPI project

What we did. We created the application's entry point, `app/main.py`. Think of it as the main entrance to AuthShield: every request enters here.

The code (your actual `app/main.py`):

```
from fastapi import FastAPI
from app.api.v1.router import api_router

app = FastAPI()

app.include_router(
    api_router,
    prefix="/api/v1"
)

@app.get("/")
async def message():
    return {
        "status":"Online",
        "message":"The Backend is working properly"
    }
```

Line by line.

`from fastapi import FastAPI` imports the FastAPI class. A class is a blueprint; you create objects from blueprints.

`app = FastAPI()` creates your application object. From this line on, `app` *is* your backend. When you run `uvicorn app.main:app --reload`, uvicorn (the web server) finds this variable and hands incoming requests to it.

`app.include_router(api_router, prefix="/api/v1")` plugs a group of routes (built in Step 2) into the application, all starting with `/api/v1`.

`@app.get("/")` is a **decorator**: a line starting with `@` that changes the function below it. This one tells FastAPI "when a GET request arrives for `/`, run the function below." GET is the method a browser uses when it simply fetches something.

`async def message():` defines the endpoint. `async` lets the function pause while waiting (for a database, say) without freezing the whole server.

`return {...}` - whatever a FastAPI endpoint returns becomes the JSON response, with status 200 (OK) by default.

What we learned. What FastAPI is; what an application instance is; how to start a backend server; how an HTTP request reaches our application.

Check it yourself. Start the server with `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/` - you should see the JSON message. Then open `http://127.0.0.1:8000/docs`: FastAPI builds an interactive testing page from your code for free.

Step 2. Create API versioning and routing

What we did. We separated the endpoints from `main.py` by creating an API router in `app/api/v1/router.py`. Our structure became:

```
app/
├── main.py
└── api/
    └── v1/
        └── router.py
```

The code (your actual `app/api/v1/router.py`):

```
from fastapi import APIRouter
from app.api.v1.routes import admin, auth, users

api_router = APIRouter()

api_router.include_router(
    auth.router,
    prefix="/auth",
    tags=["Auth"]
)
api_router.include_router(
    admin.router,
    prefix="/admin",
    tags=["Admin"]
)
api_router.include_router(
    users.router,
    prefix="/users",
    tags=["Users"]
)
```

Line by line.

`api_router = APIRouter()` creates a mini-application. It collects routes exactly like `app` does, but it cannot run alone; it must be plugged into the real app.

`api_router.include_router(auth.router, prefix="/auth", tags=["Auth"])` mounts every route from `routes/auth.py` under `/auth`. Combined with the `/api/v1` prefix from `main.py`, final addresses become `/api/v1/auth/...`. The `tags` only control grouping on the `/docs` page.

Why. Without a router, every endpoint would live in `main.py`, and the file would become unmanageable as the project grows. The `v1` in the path is **API versioning**: if you ever rebuild the API in an incompatible way, you create `v2` and old clients keep working on `v1`.

The result is a tidy address system:

```
main.py prefix     router prefix     route path      final URL
/api/v1          + /auth           + /register   =  /api/v1/auth/register
/api/v1          + /auth           + /login      =  /api/v1/auth/login
/api/v1          + /users          + /me         =  /api/v1/users/me
```

What we learned. APIRouter; include_router(); API prefixes; basic API versioning; organizing a growing backend.

Phase 2: Building the security foundation

Step 3. Create the security module

What we did. We created `app/core/security.py` to keep all security work in one place, and we started with the most important rule in authentication: **never store plain-text passwords**.

Hashing is a one-way transformation. Give it `"mypassword123"` and it produces a long scrambled string. Three properties make it useful:

1. A login attempt can be checked against the stored hash, but the hash cannot be turned back into the password.
2. A tiny change in the password produces a completely different hash.
3. It is deliberately slow, so attackers cannot try billions of guesses per second.

Hashing is not encryption. Encryption is a two-way lock that can be opened with a key. Hashing is a paper shredder: the original is gone, but you can shred a new attempt and compare the shreds.

The code (the hashing part of your `app/core/security.py`):

```
from pwdlib import PasswordHash

password_hash = PasswordHash.recommended()

def hash_password(password:str) -> str :
    return password_hash.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return password_hash.verify(plain_password, hashed_password)
```

Line by line.

`password_hash = PasswordHash.recommended()` creates a hasher using the library's current best-practice algorithm. Using `recommended()` is a quiet win: when the industry moves to a stronger algorithm, upgrading the library upgrades you.

`hash_password(password)` turns plain text into a hash. You call it exactly once per password: at registration (and later, at password reset).

`verify_password(plain_password, hashed_password)` checks a login attempt against the stored hash and returns `True` or `False`. Notice what never happens: you never compare passwords directly, and you never "un-hash" anything.

What we learned. Password hashing; password verification; why hashing differs from encryption; why passwords must not be stored in plain text; why security logic is separated from API routes. Important habit: we built these functions *before* registration existed - build the tool, then build the thing that uses the tool.

Step 4. Add environment variables and JWT configuration

What we did. We prepared `security.py` for token work by adding imports and configuration, with the secret in `.env`.

The code (the configuration part of your `security.py`):

```
import os
import jwt
from datetime import datetime, timedelta, timezone
from dotenv import load_dotenv
from uuid import uuid4

load_dotenv()
SECRET_KEY = os.getenv("SECRET_KEY")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MIMUTES = 30
REFRESH_TOKEN_EXPIRE_DAYS = 7
```

Line by line.

`load_dotenv()` reads your `.env` file into the environment, and `os.getenv("SECRET_KEY")` fetches the secret from there. The rule: **secrets live in `.env`, never in source code**, and `.env` stays out of git. Source code gets shared and committed; secrets must not.

`ALGORITHM = "HS256"` is the signing algorithm: one shared secret key both signs and verifies tokens. That is right for a single server.

The two expiry constants control token lifetimes: 30 minutes for access tokens, 7 days for refresh tokens. Step 16 explains why the lifetimes differ. Small note: `MIMUTES` is a typo of `MINUTES` in your code. It works because Python only needs the name used consistently; fix it later in one pass, renaming everywhere at once.

What we learned. Environment variables; JWT signing; token expiration; UTC time; UUIDs; configuration versus application logic.

Step 5. Implement access-token creation

What we did. We wrote the function that creates signed proof of identity: `create_access_token`.

What is a JWT? A JSON Web Token is a signed note with three dot-separated parts:

```
eyJ0eXAiOi...  .  eyJzdWIiOiIyYm...  .  SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV
   header           payload (claims)         signature
```

The payload is **readable by anyone** - it is encoded, not encrypted. The signature only proves your server wrote it and that nobody changed a character since. So never put passwords or secrets inside a token.

The code (your actual function):

```python
def create_access_token(user_id:str, session_id:str) -> str :
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MIMUTES
    )

    payload = {
        "sub": user_id,
        "exp": expire,
        "type": "access",
        "jti": str(uuid4()),
        "sid": session_id
    }
    token = jwt.encode(
        payload,
        SECRET_KEY,
        algorithm=ALGORITHM
    )
    return token
```

Line by line.

`expire = datetime.now(timezone.utc) + timedelta(minutes=30)` - “30 minutes from right now.” Always use UTC inside a server; timezones are a display problem, not a storage problem.

The payload holds five **claims**:

Claim	Meaning	Why it is there
`sub`	subject: the user ID	answers “who is this token about?”
`exp`	expiration time	a stolen token dies on its own
`type`	`"access"`	stops a refresh token being used as an access token
`jti`	unique ID of this exact token	lets you revoke one specific token (logout uses this)
`sid`	the session ID	ties the token to one login on one device

`jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)` builds the token string and signs it. Change one character of the payload and the signature check fails.

What we learned. JWT payloads and claims; the meaning of `sub`, `exp`, `jti`, `sid`; creating signed tokens; why tokens expire.

Step 6. Implement access-token decoding

What we did. Creating a token is only half the job. Every protected endpoint must be able to verify one. That second half is `decode_access_token`.

The code (your actual function):

```python
def decode_access_token(token:str) -> dict:
    payload = jwt.decode(
        token,
        SECRET_KEY,
        algorithms=[ALGORITHM]
    )
    if payload.get("type") != "access":
        raise ValueError("Invalid token type")
    return payload
```

Line by line.

`jwt.decode(...)` performs three checks in one call: the signature is genuine, the token is not expired (`exp` is checked automatically), and the algorithm is the one you allow. Any failure raises an exception.

`if payload.get("type") != "access"` adds your own fourth check: the token must be an *access* token. Without this, a refresh token (same signature, same secret) could be replayed against protected endpoints.

What we learned. Token validation; JWT decoding; signature verification; expiration checking; token-type validation.

Your security foundation is now complete: hashing, verification, token creation, token validation. Everything after this point reuses these four tools.

Phase 3: Creating users and their storage

Step 7. Create the user repository

What we did. We needed somewhere to store users, so we created `app/repositories/user_repository.py` using a Python dictionary instead of a real database.

Why a repository? Your security module handles passwords and tokens; it should not also manage user records. The **repository pattern** gives data its own home: one file that knows how to save, find, and update users. Everything else asks the repository instead of touching storage directly. Today the storage is a dictionary; in Stage 4 it becomes PostgreSQL. Because the rest of the code only talks to repository functions, that future swap changes one file instead of the whole project.

The code (your actual `app/repositories/user_repository.py`):

```
import uuid

users = {}

async def get_user_by_email(email:str):
    for user in users.values():
        if user["email"] == email:
            return user
    return None

async def get_user_by_id(user_id:str):
    for user in users.values():
        if user["id"] == user_id:
            return user
    return None

async def create_user(user_data: dict):
    user_id = str(uuid.uuid4())
    new_user = {
        "id": user_id,
        "username": user_data["username"],
        "email": user_data["email"],
        "hashed_password": user_data["hashed_password"],
        "is_email_verified": False
    }
    users[user_id] = new_user
    return new_user

async def verify_user_email(user_id:str):
    user = users.get(user_id)
    if user is None:
        return None
    user["is_email_verified"] = True
```

Line by line.

`users = {}` - the whole “database.” A dictionary maps keys to values; here each key is a user ID and each value is the user’s record.

`get_user_by_email(email)` loops through every stored user and returns the first match, or `None`. Login uses this to find you. A full scan is fine for learning and slow with a million users; a real database answers this with an index.

`create_user(user_data)` builds the record. `uuid.uuid4()` generates a random ID like `2ba23367-cc55-...`. Random IDs cannot be guessed, unlike counting IDs (1, 2, 3…), which invite snooping. New users start with `is_email_verified: False` - unverified until proven otherwise (Step 19 uses this flag).

`verify_user_email(user_id)` flips the flag after the user proves they own the email address.

What we learned. Repository pattern; CRUD operations; looking up users by email or ID; separating storage from security logic; using in-memory dictionaries for early development.

The honest limitation. This dictionary lives in the server’s memory. Restart the server and every user vanishes; run two server workers and each keeps its own separate dictionary. For learning, that is perfect: zero setup, instant reset. PostgreSQL fixes it in Stage 4.

Phase 4: Registration and login

Step 8. Create the authentication service

What we did. With password functions and a user repository ready, we connected them in `app/services/auth_service.py`, starting with `register_user`.

Why a service? The route speaks HTTP; the service speaks authentication. The route reads the request body and sends responses; the service coordinates the actual workflow. Each layer has one job, so when something breaks you know where to look.

```
Route receives registration request
              |
              v
Service checks for existing user
              |
              v
Security module hashes password
              |
              v
Repository stores user
              |
              v
Service returns result
              |
              v
Route sends HTTP response
```

The code (your actual registration service):

```python
async def register_user(user_data:dict):
    existing = await get_user_by_email(user_data["email"])
    if existing is not None:
        raise ValueError("Email already registered")

    hashed_password = hash_password(user_data["password"])

    new_user_data = {
        "username": user_data["username"],
        "email": user_data["email"],
        "hashed_password": hashed_password
    }

    new_user = await create_user(new_user_data)

    verification_token = create_email_verification_token(
        str(new_user["id"])
    )

    return {
        "id": new_user["id"],
        "username": new_user["username"],
        "email": new_user["email"],
        "verification_token": verification_token
    }
```

Line by line.

`existing = await get_user_by_email(...)` - the duplicate check comes first. Never create two accounts for one email.

`hash_password(...)` - the plain password is used here and never stored. Only the hash is kept.

`create_user(new_user_data)` - the repository stores the record.

`create_email_verification_token(...)` - a signed token that proves “the person holding this owns this email address.” Step 21 covers the full mechanism.

The return value never includes the password or its hash. The verification token is returned in the response for learning convenience; in production you would email a link instead.

What we learned. Business logic; service functions; calling functions across modules; the difference between routes, services, repositories, and security utilities.

Step 9. Create the authentication schemas

What we did. We defined what valid requests look like using Pydantic models in `app/schemas/auth.py`.

The code (your actual schemas):

```
from pydantic import BaseModel, Field, EmailStr

class UserCreate(BaseModel):
    username: str
    email: EmailStr
    password: str = Field(min_length=8, max_length=20)

class RefreshTokenRequest(BaseModel):
    refresh_token: str

class ForgotPasswordRequest(BaseModel):
    email: EmailStr

class ResetPasswordRequest(BaseModel):
    token: str
    new_password: str = Field(min_length=8)
```

Line by line.

`class UserCreate(BaseModel)` describes the shape of a registration request. `EmailStr` is not just a string: Pydantic checks it actually looks like an email address. `Field(min_length=8)` rejects short passwords automatically, before your code runs.

Why. Without schemas, you would accept arbitrary request data and hand-check every field. Pydantic validates incoming data before your service ever sees it, and FastAPI turns validation failures into clear error responses for free.

An honest status note. Your `register` and `login` routes currently accept a plain `dict`, so `UserCreate` is written but not yet enforced. Switching the route to `data: UserCreate` is a one-line change that turns all these rules on - a good first exercise in Stage 1, item 5.

What we learned. Pydantic; BaseModel; request-body validation; required fields; separating API input structures from internal data.

Step 10. Create the authentication routes

What we did. We connected the services to real HTTP endpoints in `app/api/v1/routes/auth.py`, starting with register and login.

The code (your actual register and login routes):

```python
@router.post("/register")
async def register(data:dict):
    try:
        user = await register_user(data)
        return user
    except ValueError as error:
        raise HTTPException(
            status_code=400,
            detail=str(error)
        )

@router.post("/login")
async def login(data:dict):
    try:
        user = await login_user(
            data["email"],
            data["password"]
            )
    except ValueError as error:
        raise HTTPException(
            status_code=401,
            detail=str(error)
        )
    return user
```

Line by line.

`@router.post("/register")` - same decorator idea as Step 1, now on the router: POST requests to `/api/v1/auth/register` run this function.

`try / except ValueError` - the service signals expected failures (duplicate email, wrong password) by raising `ValueError` with a readable message. The route converts that into an HTTP error: 400 for a bad registration, 401 for a failed login. This split is the route's real job: the service decides *what* went wrong, the route decides *how HTTP should say it*.

And the login service itself (your actual code):

```python
async def login_user(email: str, password: str):
    user = await get_user_by_email(email)
    if user is None:
        raise ValueError("Invalid email or password")

    password_is_correct = verify_password(
        password,
        user["hashed_password"]
    )
    if not password_is_correct:
        raise ValueError("Invalid email or password")

    if not user["is_email_verified"]:
        raise ValueError("Please verify your email before logging in")

    session_id = str(uuid4())
    create_session(
        session_id=session_id,
        user_id=str(user["id"])
    )

    access_token = create_access_token(
        user_id=str(user["id"]),
        session_id=session_id
    )
    refresh_token = create_refresh_token(
        user_id=str(user["id"]),
        session_id=session_id
    )

    refresh_payload = decode_refresh_token(refresh_token)
    refresh_token_id = refresh_payload["jti"]

    set_refresh_token_id(
        session_id=session_id,
        token_id=refresh_token_id
    )

    return {
        "id": user["id"],
        "username": user["username"],
        "email": user["email"],
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer"
    }
```

Line by line.

The error message `"Invalid email or password"` is identical for a missing account and a wrong password. That is deliberate and good security: it never tells an attacker which part failed, so they cannot probe which emails are registered.

`session_id = str(uuid4())` then `create_session(...)` - every login gets a fresh session (Step 13 explains what a session stores).

`decode_refresh_token(refresh_token)` right after creating the token looks odd: you just made it, why decode it? Because the token's unique `jti` was generated *inside* `create_refresh_token`, and you need that ID to store in the session. Decoding is how you read it back out.

`set_refresh_token_id(...)` - the session now knows which refresh token is current. This is the anchor for rotation in Step 18.

What we learned. FastAPI route handlers; POST requests; request bodies; HTTPException; HTTP status codes; connecting services to routes; testing APIs with Postman.

Phase 5: Protecting endpoints

Step 11. Create the dependency module

What we did. After login worked, we needed a guard so only authenticated users can reach certain endpoints. That guard lives in `app/api/v1/dependencies.py`.

The code (your actual dependency):

```
oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="/api/v1/auth/login"
)

async def get_current_user(token:str = Depends(oauth2_scheme)):
    try:
        payload = decode_access_token(token)
    except Exception:
        raise HTTPException(
            status_code=401,
            detail="Invalid or expired access token",
            header={"WWW-Authenticate":"Bearer"}
        )

    user_id = payload.get("sub")
    token_id = payload.get("jti")
    session_id = payload.get("sid")

    if not user_id or not token_id or not session_id:
        raise HTTPException(status_code=401, detail="Invalid token payload", ...)

    if is_token_revoked(token_id):
        raise HTTPException(status_code=401, detail="Session is invalid or revoke", ...)

    user = await get_user_by_id(user_id)
    if user is None:
        raise HTTPException(status_code=401, detail="User not found", ...)
    return user
```

Line by line.

`OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")` creates a helper that extracts the token from the request's `Authorization: Bearer eyJ...` header. It does not validate anything itself; the `tokenUrl` mainly tells the `/docs` page where to log in. Validation is the next function's job.

`Depends(oauth2_scheme)` - **dependency injection**: FastAPI runs `oauth2_scheme` first and passes its return value (the token) into `get_current_user`.

Then the checks, each existing for a reason:

1. **Decode** - is the signature genuine and unexpired? (Stops forgeries and old tokens.)
2. **Claims present** - does the token carry `sub`, `jti`, `sid`? (Stops malformed tokens.)
3. **Token revoked?** - is this exact token ID in the revoked set? (This is how logout kills a token; Step 14.)
4. **User exists?** - the token is only as good as the account behind it.

What we learned. Depends(); dependency injection; OAuth2PasswordBearer; Bearer tokens; authentication dependencies; protecting routes.

A gap found by testing. The guard checks whether the *token* was revoked, but not whether the *session* was revoked. So killing a session did not block its access tokens. The final part of this book shows the tested fix.

Step 12. Create the user routes

What we did. We created the first protected endpoint, `GET /api/v1/users/me`, in `app/api/v1/routes/users.py`.

The code (your actual route):

```python
@router.get("/me")
async def read_current_user(current_user = Depends(get_current_user)):
    ...
```

(Your route returns the user's id, username, and email from `current_user`.)

Line by line.

`Depends(get_current_user)` means the endpoint function only ever runs for requests that survived the full guard from Step 11 - and it receives the already-verified user as a ready-made argument. The endpoint itself stays almost empty, which is the sign of a good guard.

What happens when the endpoint is called:

```
Client sends access token
            |
            v
OAuth2PasswordBearer extracts the token
            |
            v
get_current_user() validates it
            |
            v
Backend retrieves the user
            |
            v
/users/me returns that user's information
```

Check it yourself (the same Postman steps you used). Log in and copy the access token. Open `GET /api/v1/users/me`, go to the Authorization tab, choose Bearer Token, paste, send. You get your own user record back. Send the same request without the token and you get a 401 - that contrast is the whole point of a protected route.

What we learned. Protected endpoints; authentication versus public routes; how a token identifies a user; how FastAPI dependencies pass data to endpoints.

Phase 6: Managing sessions and logout

Step 13. Create the session repository

What we did. We learned that a user and a session are not the same thing, and gave sessions their own home in `app/repositories/session_repository.py`.

Why. A user can log in on a laptop and a phone. Those are two separate sessions:

```
User
 ├── Laptop session
 └── Phone session
```

A session ID lets the backend tell these logins apart, so it can revoke one device's login without touching the other. This is also what makes "log out everywhere after a password reset" possible later.

The code (your actual session store):

```python
sessions = {}

def create_session(session_id:str, user_id:str):
    sessions[session_id] = {
        "user_id": user_id,
        "revoked": False,
        "refresh_token_id": None,
        "refresh_token_history": []
    }

def get_session(session_id:str):
    return sessions.get(session_id)

def revoke_session(session_id:str):
    session = sessions.get(session_id)
    if session:
        session["revoked"] = True
```

Line by line.

Each session records four facts: who owns it (`user_id`), whether it has been killed (`revoked`), which refresh token is currently valid for it (`refresh_token_id`), and which refresh tokens were used before (`refresh_token_history`). Step 18 shows why the history matters.

`revoke_session` does not delete the session; it flips a flag. Keeping the record matters: a deleted session is indistinguishable from a typo, while a revoked session is a deliberate "this login is over."

What we learned. Sessions; session IDs; multiple device logins; session revocation; the relationship between users, tokens, and sessions.

Step 14. Create the token-revocation repository

What we did. We created `app/repositories/token_repository.py` to remember which access tokens have been revoked.

Why. A JWT stays cryptographically valid until it expires; you cannot "unsign" it. Logout therefore needs a **denylist**: a set of token IDs the server refuses to honor even though they would otherwise still be valid. The guard from Step 11 checks this set on every protected request.

The code (your actual repository, complete):

```python
revoked_token = set()

def revoke_token(token_id:str) -> None:
    revoked_token.add(token_id)

def is_token_revoked(token_id:str) -> bool:
    return token_id in revoked_token
```

Line by line.

A `set` is the right shape: it stores unique items, and the `in` check stays fast no matter how many tokens it holds. `revoke_token` adds a token's `jti`; `is_token_revoked` answers yes or no. This tiny file is the entire mechanism behind logout - and, later, behind single-use reset

tokens.

Limitation. An in-memory set disappears on restart, so a server restart also forgets every logout. Production needs persistent or shared revocation state (Stage 4).

What we learned. Token revocation; the purpose of `jti`; why JWT expiration alone does not provide immediate logout; checking token status during authentication.

Step 15. Implement logout

What we did. We added `POST /api/v1/auth/logout` to `routes/auth.py`.

The code (your actual route):

```
@router.post("/logout")
async def logout(
    token:str = Depends(oauth2_scheme),
    current_user:dict = Depends(get_current_user)):

    payload = decode_access_token(token)
    token_id = payload.get("jti")
    session_id = payload["sid"]

    revoke_token(token_id)
    revoke_session(session_id)

    return {
        "message": "Successfully logged out"
    }
```

Line by line.

The route takes two dependencies: the raw token (so it can read the IDs) and the current user (so only a valid logged-in request can log out). Then two actions: `revoke_token(token_id)` kills this exact access token, and `revoke_session(session_id)` ends the whole login.

Why both? The token set tracks dead access tokens; the session tracks the login itself. Revoking the session also stops the refresh token from minting new access tokens - otherwise logout would be undone at the next refresh.

What we learned. Logout implementation; access-token revocation; session revocation; coordinating multiple repositories.

At this point the project had registration, login, protected routes, sessions, and logout - a complete basic authentication system.

Phase 7: Refresh tokens and rotation

Step 16. Add refresh-token creation and decoding

What we did. We extended `security.py` with `create_refresh_token` and `decode_refresh_token`.

Why two kinds of token? An access token is shown on every request, so it is the easiest to steal; keeping it short-lived (30 minutes) limits the damage. But logging in every 30 minutes is a terrible experience. The refresh token solves this: it lives 7 days and is used only to ask for new tokens. Think of the access token as a day pass and the refresh token as the ID card you show at the gate to get a new day pass.

The code (your actual functions, shortened to the differences):

```
def create_refresh_token(user_id:str, session_id:str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        days=REFRESH_TOKEN_EXPIRE_DAYS
    )
    payload = {
        "sub": user_id,
        "exp": expire,
        "type": "refresh",
        "jti": str(uuid4()),
        "sid": session_id
    }
    ...

def decode_refresh_token(token:str) -> dict:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    if payload.get("type") != "refresh":
        raise ValueError("Invalid token type")
    return payload
```

Line by line. Same machinery as the access token, with two deliberate differences: the lifetime is 7 days instead of 30 minutes, and `type` is `"refresh"`. The type check in the decoder is what stops a refresh token from being replayed as an access token, and vice versa.

What we learned. Access token versus refresh token; different token lifetimes; separate token types; refresh-token validation.

Step 17. Implement the refresh endpoint

What we did. We added `POST /api/v1/auth/refresh` to `routes/auth.py`.

Why. The client can get new tokens without resubmitting the password, as long as the refresh token and the session remain valid.

The code (your actual route, condensed):

```
@router.post("/refresh")
async def refresh_access_token(data: RefreshTokenRequest):
    try:
        payload = decode_refresh_token(data.refresh_token)
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid or expired refresh token")

    user_id = payload.get("sub")
    session_id = payload.get("sid")
    token_id = payload.get("jti")

    session = get_session(session_id)
    if session is None or session["revoked"]:
        raise HTTPException(status_code=401, detail="Session is invalid or revoked")
    ...
```

Line by line. The checks happen in a careful order. Decode first (is it even a real, unexpired token?). Then session status (is this login still alive?). Only then consider issuing anything. Note the route uses your `RefreshTokenRequest` schema from Step 9 - Pydantic guarantees `data.refresh_token` exists before this code runs.

What we learned. Refresh workflows; token validation; session checks; reusing service and repository functions across endpoints.

Step 18. Implement refresh-token rotation

What we did. We improved the refresh endpoint so every successful refresh issues a *new* refresh token and retires the old one. This is **rotation**.

Why. If an attacker steals a refresh token, rotation turns silent theft into a loud alarm. Either the thief uses the token first (and your next legitimate refresh suddenly fails, because the session moved on) or you use yours first (and the thief's copy becomes a known-used token). Both outcomes expose the theft.

The code (your session history functions in `session_repository.py`):

```
def set_refresh_token_id(session_id:str, token_id:str):
    session = sessions.get(session_id)
    if session:
        old_token_id = session["refresh_token_id"]
    if old_token_id is not None:
        session["refresh_token_history"].append(old_token_id)
    session["refresh_token_id"] = token_id

def was_refresh_token_used(session_id:str, token_id:str) -> bool:
    session = sessions.get(session_id)
    if session is None:
        return False
    return token_id in session["refresh_token_history"]
```

And the rotation checks inside the refresh route:

```
    if was_refresh_token_used(session_id, token_id):
        revoke_session(session_id)
        raise HTTPException(status_code=401,
            detail="Refresh token reuse detected. Session revoked;please log in again")

    if session["refresh_token_id"] != token_id:
        raise HTTPException(status_code=401, detail="User not found")
```

Line by line.

`set_refresh_token_id` archives the previous token ID into `refresh_token_history` before installing the new one. `was_refresh_token_used` answers the alarm question: "has this exact token already been spent?"

The route then enforces the rule: reuse a spent token and the *entire session* is revoked - treat it as theft. A token that is neither current nor spent is also rejected. Only the session's current token can mint new tokens.

A rough edge worth knowing. The second check replies `"User not found"`, which is the wrong message - the real problem is a stale token. Small message fixes belong to Stage 1, item 5.

What we learned. Refresh-token rotation; reuse detection; token history; why refresh-token theft is a security concern. Production rotation additionally needs careful handling of concurrent requests and persistent storage (Stage 4).

Phase 8: Email verification

Step 19. Add email-verification status to users

What we did. Registration already creates users with `is_email_verified: False`, and `verify_user_email(user_id)` flips the flag (both in Step 7's repository code).

Why. Separating registration from verification means an account exists in a limited state until the user proves they own the email address. Login enforces that state: unverified users are told to verify first.

What we learned. Account status flags; boolean fields; separating registration from verification.

Step 20. Add email-verification token functions

What we did. We added `create_email_verification_token` and `decode_email_verification_token` to `security.py`.

The code (your actual functions, condensed):

```
def create_email_verification_token(user_id:str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=15)
    payload = {
        "sub": user_id,
        "exp": expire,
        "type": "email_verification",
        "jti": str(uuid4())
    }
    ...

def decode_email_verification_token(token:str) -> dict:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    if payload.get("type") != "email_verification":
        raise ValueError("Invalid email verification token")
    return payload
```

Why a separate token type? An email-verification token has one job: prove email ownership for 15 minutes. The type check means an access token (same signing secret) cannot be substituted for it. Every token in AuthShield carries a `type`, and every decoder demands exactly one type. That pattern is the real lesson of this step.

Bugs you fixed here. Your lesson notes record two real debugging wins: `algorithm=` had to be `algorithms=` (PyJWT wants a list), and `"email_verfication"` had to be `"email_verification"` (exact strings matter when comparing token types). These are the classic JWT mistakes; hitting them yourself is how they stick.

What we learned. Purpose-specific tokens; token expiration; token-type validation; debugging JWT decoding errors.

Step 21. Connect email verification to registration

What we did. Registration now generates a verification token (Step 8's service code), and we added `POST /api/v1/auth/verify-email`.

The code (your actual route):

```python
@router.post("/verify-email")
async def verify_email(data: RefreshTokenRequest):
    try:
        payload = decode_email_verification_token(data.refresh_token)
        user_id = payload.get("sub")
        if not user_id:
            raise HTTPException(status_code=401, detial="Invalid verification token")

        user = await verify_user_email(user_id)
        if user is None:
            raise HTTPException(status_code=401, detail="User not found")

        return {
            "message": "Email verified succesfully",
            "is_email_verified": user["is_email_verified"]
        }
    except HTTPException:
        raise
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid or expired verification token")
```

Line by line.

`except HTTPException: raise` matters: it lets your deliberate 401s pass through untouched, while the final `except Exception` converts any decode failure into a clean 401 instead of an ugly server crash.

Two honest notes. There is a typo, `detial=` instead of `detail=`, in the missing-user-ID branch: if that branch ever ran, FastAPI would crash with a 500 instead of returning your message - a real bug, though rarely reached. And the route borrows `RefreshTokenRequest` because it happens to have one string field; it works, but the field is named `refresh_token` even though you send a verification token. A dedicated schema would say what it means - naming is documentation.

Check it yourself (your Postman sequence). Register a user, copy the verification token from the response, submit it to `/verify-email`, confirm `is_email_verified` becomes `true`, then log in. Each of these was re-run while writing this book and behaves exactly as described.

Current limitations. The token comes back in the API response instead of an email (fine for learning, wrong for production), and it is reusable within its 15 minutes - harmless once the account is verified, but single-use verification tokens are Stage 1, item 3, and the fix works exactly like the reset-token fix in the final part of this book.

Phase 9: Password reset

Step 22. Add password-reset token functions

What we did. We added `create_password_reset_token` and `decode_password_reset_token` to `security.py`.

The code (your actual functions, condensed):

```
def create_password_reset_token(user_id:str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=15)
    payload = {
        "sub": user_id,
        "exp": expire,
        "type": "password_reset",
        "jti": str(uuid4())
    }
    ...

def decode_password_reset_token(token:str) -> dict:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    if payload.get("type") != "password_reset":
        raise ValueError("Inavlid password reset token")
    return payload
```

Why. A user who forgets their password needs a secure way to authorize a password change without knowing the old one. The reset token is that temporary authorization: signed, 15-minute lifetime, its own token type so no other token can be substituted.

Step 23. Add password-reset business logic

What we did. We added `forgot_password` and `reset_password` to `auth_service.py`.

The code (your actual functions):

```
async def forgot_password(email:str):
    user = await get_user_by_email(email)
    if user is None:
        raise ValueError("User not Found")
    token = create_password_reset_token(str(user["id"]))
    return {
        "message": "password reset token generated",
        "reset_token": token
    }

async def reset_password(token:str , new_password:str) :
    try:
        payload = decode_password_reset_token(token)
    except Exception:
        raise ValueError("Invalid or expired password reset token")

    user_id = payload.get("sub")
    if not user_id:
        raise ValueError("Invalid password reset token")

    user = await get_user_by_id(user_id)
    if user is None:
        raise ValueError("User Not found")
    user["hashed_password"] = hash_password(new_password)

    return {
        "message": "Password has reset successfully"
    }
```

Line by line.

`forgot_password` finds the user and mints a reset token. One information leak to know about: `"User not Found"` tells an attacker which emails are registered - the very thing login carefully hides. Production systems always answer “if this email exists, we sent a link.” That fix belongs to Stage 1, item 5.

`reset_password` decodes the token, finds the user from its `sub`, and replaces the stored hash. Notice the password is hashed with the *same* `hash_password` from Step 3 - one tool, reused everywhere.

Why the service? The service coordinates: the security module owns tokens, the repository owns user records, and the service orchestrates both. Nothing new to learn, everything reused - a sign the architecture is working.

Step 24. Add password-reset request schemas

What we did. We added `ForgotPasswordRequest` and `ResetPasswordRequest` to `schemas/auth.py` (shown in Step 9).

Why. The routes read `data.email`, `data.token`, and `data.new_password` knowing Pydantic has already guaranteed they exist, that the email looks like an email, and that the new password is at least 8 characters. Schemas are the API's contract, checked before your code runs.

Step 25. Add password-reset routes

What we did. We added the two endpoints to `routes/auth.py`.

The code (your actual routes):

```
@router.post("/forgot-password")
async def forgot_password_route(data: ForgotPasswordRequest):
    try:
        return await forgot_password(data.email)
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error))

@router.post("/reset-password")
async def reset_password_route(data: ResetPasswordRequest):
    try:
        return await reset_password(
            token=data.token,
            new_password=data.new_password
        )
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error))
```

Check it yourself (your Postman sequence).

1. `POST /api/v1/auth/forgot-password` with `{"email": "user@example.com"}` - copy the reset token.
2. `POST /api/v1/auth/reset-password` with `{"token": "PASTE_TOKEN", "new_password": "NewPassword123!"}` - should succeed.
3. Log in with the new password - confirms the updated hash flows through the normal login.

What remains incomplete (your own list, confirmed by testing). Single-use reset tokens; session revocation after a reset; friendlier handling of unknown emails; rate limiting; email delivery instead of returning tokens. The first two are fixed, with tested code, in the final part of this book.

Phase 10: How all the files work together

Now that the whole sequence is built, here is the map of what each file owns.

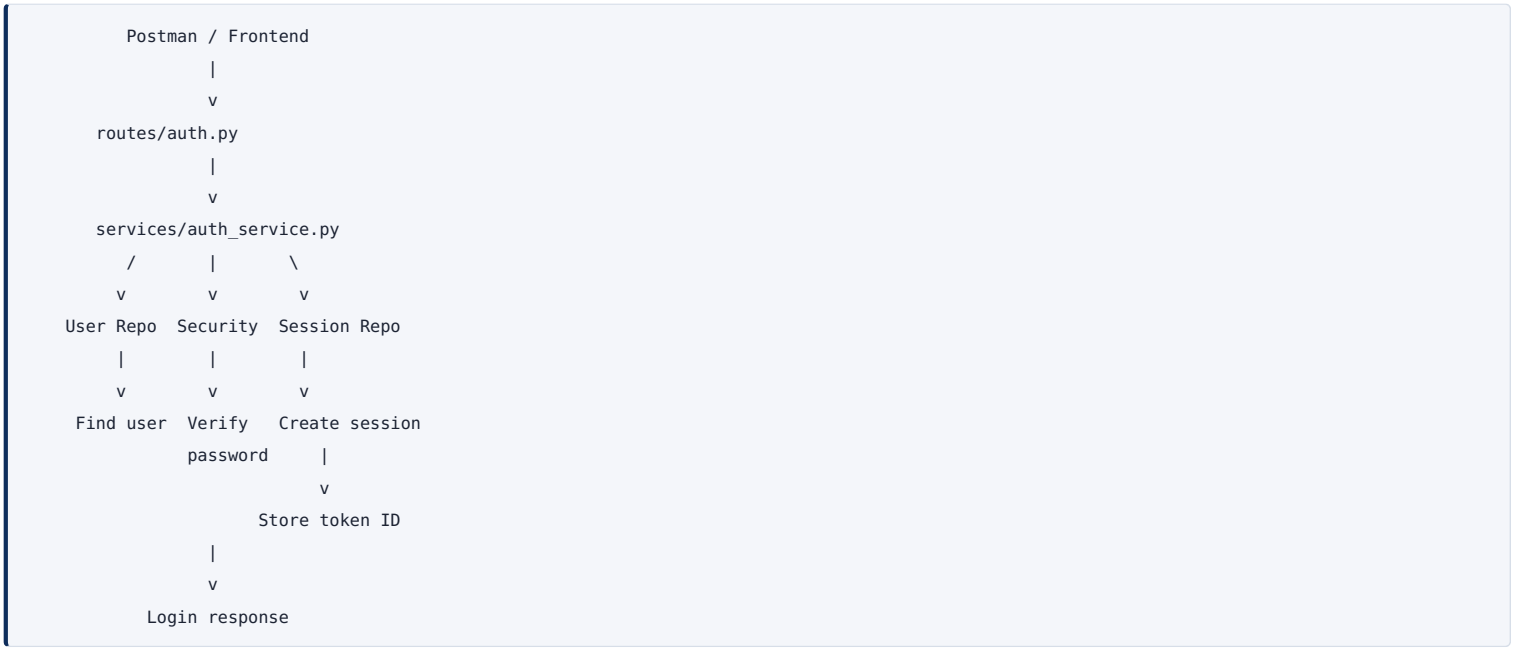
File	Owns	Key lesson
`main.py`	the application instance	every request enters here
`api/v1/router.py`	mounting route groups	prefixes and versioning
`routes/auth.py`	auth endpoints	HTTP in, HTTP out
`routes/users.py`	protected user endpoints	the guard in action
`routes/admin.py`	admin endpoints (placeholder)	Stage 2 will fill this
`dependencies.py`	the authentication guard	Depends() and validation order
`core/security.py`	hashing and tokens	build tools first, reuse them
`schemas/auth.py`	request shapes	validate at the boundary
`repositories/user_repository.py`	user records	one home per kind of data
`repositories/token_repository.py`	revoked token IDs	the denylist behind logout
`repositories/session_repository.py`	sessions and rotation state	one login per device
`services/auth_service.py`	authentication workflows	the coordinator
`.env`	secrets	never in source code

The most important idea in the whole project: each file has one responsibility, and a request travels through them in a predictable order. When something breaks, the layer tells you where to look.

Phase 11: Tracing a complete login request

Follow one login end to end, naming the file at every hop.

1. **The request enters the route** (`routes/auth.py`). The login route reads the body and calls `login_user(email, password)` .
2. **The service finds the user** (`services/auth_service.py` calls `get_user_by_email`).
3. **The repository retrieves the record** (`repositories/user_repository.py` scans the dictionary).
4. **The password is verified** (`core/security.py` : `verify_password` against the stored hash). Wrong password: `ValueError` , which the route turns into a 401.
5. **The email status is checked** - unverified accounts stop here.
6. **A session is created** (`repositories/session_repository.py` : fresh session ID, `revoked: False`).
7. **Tokens are created** (`core/security.py` : access token for 30 minutes, refresh token for 7 days, both carrying the session ID).
8. **The refresh-token ID is stored** in the session, anchoring future rotation.
9. **The response returns** through the service to the route to the client.



Registration, email verification, refresh, logout, and password reset all reuse this same architecture: route, service, repositories, security.

Phase 12: Your complete learning sequence

The whole journey in one table - the learning order in which this book presents each piece, and what it adds.

Step	File created or modified	What we implemented
1	`main.py`	FastAPI application
2	`api/v1/router.py`	API versioning and routing
3	`core/security.py`	Password hashing
4	`.env` , `core/security.py`	Secret configuration
5	`core/security.py`	Access-token creation
6	`core/security.py`	Access-token decoding
7	`repositories/user_repository.py`	User storage and lookup
8	`services/auth_service.py`	Registration logic
9	`schemas/auth.py`	Request validation
10	`routes/auth.py` , `router.py`	Registration and login endpoints
11	`api/v1/dependencies.py`	Bearer extraction and current-user guard
12	`routes/users.py`	Protected `/users/me` endpoint
13	`repositories/session_repository.py`	Session management
14	`repositories/token_repository.py`	Token revocation
15	`routes/auth.py`	Logout
16	`core/security.py`	Refresh-token creation and decoding
17	`routes/auth.py` , `schemas/auth.py`	Refresh endpoint
18	`session_repository.py` , `routes/auth.py`	Rotation and reuse detection
19	`user_repository.py`	Email-verification status
20	`core/security.py`	Email-verification tokens
21	`auth_service.py` , `routes/auth.py`	Email-verification workflow
22	`core/security.py`	Password-reset tokens
23	`auth_service.py`	Password-reset service logic
24	`schemas/auth.py`	Password-reset request schemas
25	`routes/auth.py`	Forgot/reset password endpoints

This is the reconstructed learning sequence, not a guarantee that every file changed in precisely this historical order.

Phase 13: What you learn next

Your roadmap, in the order you defined it.

Stage 1 - finish authentication security (you are here). Single-use reset tokens and session revocation after reset are done for you in the next part of this book, with tested code. Remaining: single-use verification tokens, a review of rotation under concurrency, better validation and error messages, and rate limiting.

Stage 2 - authorization. Authentication asks “who are you?”; authorization asks “what may you do?” Roles, role-based access control, admin-only endpoints (your `admin.py` placeholder is waiting), permissions, and resource ownership.

Stage 3 - automated testing. `pytest` and FastAPI’s test client. Everything in the “What was actually tested” chapter, turned into tests that run in seconds and fail loudly if anything ever breaks.

Stage 4 - PostgreSQL. Replace the dictionaries with real tables: users, sessions, revoked tokens. Data survives restarts, unique constraints enforce “one account per email” for real, and transactions make token consumption atomic.

Stage 5 - production readiness. Real email delivery, secrets management, rate limiting, HTTPS and CORS, logging, and deployment.

Part II: Verified status and the Stage 1 fixes

What was actually tested

This book separates reading from running. Reading code tells you what it *intends*; only running it tells you what it *does*. While writing this book, your app was started and exercised through its real HTTP endpoints. Here is the full record:

Test	Result	Verdict
Register a new user	200, user + verification token returned	works
Register the same email again	400 “Email already registered”	works
Login before verifying email	401 “Please verify your email before logging in”	works
Verify email with the token	200, `is_email_verified: true`	works
Login with correct password	200, access + refresh tokens returned	works
`GET /users/me` with the access token	200, user data returned	works
Refresh with the refresh token	200, new token pair returned	works
Reuse the same refresh token again	401 “Refresh token reuse detected”	works
Forgot password	200, reset token returned	works
Reset password with the token	200, password changed	works
Reset again with the SAME token	**200 - succeeded again**	**gap: token reusable**
Old session after a password reset	**200 - still worked**	**gap: sessions not revoked**

The last two rows are exactly the gaps your Stage 1 list predicted - and they are why we never call a feature done without testing it.

Stage 1, Fix 1: single-use reset tokens

Why. A JWT is valid until it expires; nothing in the signature knows whether it was used. Your project already owns the perfect tool for “this token is dead”: the revoked-token set from Step 14, the same one logout uses. A reset token’s `jti` gets added to that set the moment the reset succeeds.

Edit `app/services/auth_service.py`. Add one import at the top with the others:

```
from app.repositories.token_repository import revoke_token, is_token_revoked
```

Replace the existing `reset_password` function with:

```
async def reset_password(token:str , new_password:str) :
    try:
        payload = decode_password_reset_token(token)

    except Exception:
        raise ValueError("Invalid or expired password reset token")

    token_id = payload.get("jti")

    if not token_id:
        raise ValueError("Invalid password reset token")

    if is_token_revoked(token_id):
        raise ValueError("This password reset token has already been used")

    user_id = payload.get("sub")

    if not user_id:
        raise ValueError("Invalid password reset token")

    user = await get_user_by_id(user_id)

    if user is None:
        raise ValueError("User Not found")

    user["hashed_password"] = hash_password(new_password)

    revoke_token(token_id)

    return {
        "message": "Password has reset successfully"
    }
```

Line by line. The changes teach three key ideas. We read the token's unique `jti`; we reject it if it was already spent; and we spend it only *after* the password update succeeds, so a failed attempt never burns the user's link.

Stage 1, Fix 2: revoke the user's sessions after a reset

Why. If someone else is logged into your account, changing your password should end their access. That means marking every session belonging to the user as revoked.

Edit `app/repositories/session_repository.py`. Add this new function at the bottom:

```
def revoke_user_sessions(user_id:str):
    for session in sessions.values():
        if session["user_id"] == user_id:
            session["revoked"] = True
```

Edit `app/services/auth_service.py`. Add `revoke_user_sessions` to the session imports, then add one line at the end of `reset_password`, right after `revoke_token(token_id)`:

```
    revoke_token(token_id)
    revoke_user_sessions(user_id)
```

The discovery testing forced on us

Here is the valuable part. After both fixes were applied, the tests showed:

- second use of the reset token: **400 - correctly rejected**
- login with the new password: **200 - works**
- old session's access token after the reset: **200 - STILL WORKED**

Revoking the session did nothing for access tokens. Why? Look back at the guard from Step 11: it checks whether the *token* is in the revoked set, but it never asks about the *session*. The session's `revoked` flag existed, yet nothing on the protected-route path ever read it.

Fix 3: make the guard check the session. Edit `app/api/v1/dependencies.py`. Add the import:

```
from app.repositories.session_repository import get_session
```

Then add this check in `get_current_user`, right after the `is_token_revoked` block:

```
    session = get_session(session_id)
    if session is None or session["revoked"]:
        raise HTTPException(
            status_code=401,
            detail="Session is invalid or revoked",
            headers={"WWW-Authenticate":"Bearer"}
        )
```

Now the test record reads:

- second use of the reset token: **400 - rejected**
- old session's access token after the reset: **401 - rejected**
- fresh login with the new password: **200 - works**

That is the complete loop for this reset-security exercise: find the gap by testing, fix the smallest thing that closes it, test again. The remaining Stage 1 items - single-use verification tokens, better validation and error messages, rate limiting - are still ahead of you, and each will follow this same loop.

An honest limit to keep in mind. The check-then-mark pattern above ("is it used? … then mark it") is correct for your single-process learning server. At production scale, two requests can arrive at the same instant, both pass the check, and both use the token. Databases solve this with atomic operations - a single "insert only if absent" that cannot interleave. So when you reach Stage 4, moving the revoked set into PostgreSQL is not only about surviving restarts; you will also re-express this check as one atomic insert. You do not need to solve that today. You need to know it exists.

Appendix A: References for going deeper

These are the official references for the tools in this project - the same sources used to ground this book's explanations of library behavior.

- FastAPI (routing, dependencies, security, testing): https://fastapi.tiangolo.com/tutorial/ and https://fastapi.tiangolo.com/tutorial/testing/
- PyJWT (encoding, decoding, expiration, algorithms): https://pyjwt.readthedocs.io/
- pwdlib (password hashing, `PasswordHash.recommended()`): https://frankie567.github.io/pwdlib/
- Pydantic (models, validation, `Field`): https://docs.pydantic.dev/
- python-dotenv (loading `.env` files): https://saurabh-kumar.com/python-dotenv/
- JWT concepts and the token structure: https://www.jwt.io/introduction

Appendix B: Glossary

Access token - short-lived signed proof of identity, sent with every request. **Claim** - a statement inside a JWT, like `sub` or `exp`. **Decorator** - a `@` line that changes the function below it. **Dependency injection** - FastAPI running a helper and passing its result into your function. **Denylist** - the set of revoked token IDs the server refuses to honor. **Hashing** - a one-way transformation used to store passwords safely. **JWT** - a signed token with three parts: header, payload, signature. **Refresh token** - long-lived token used only to obtain new access tokens. **Repository** - the one file that owns storage for a kind of record. **Rotation** - replacing the refresh token on every use, so theft becomes visible. **Session** - one login on one device, tracked by the server. **Service** - the layer that owns business logic between routes and repositories.

Appendix C: Postman recipes

All requests go to `http://127.0.0.1:8000` with header `Content-Type: application/json`.

Register: `POST /api/v1/auth/register` `{"username": "rishi", "email": "you@example.com", "password": "password123"}`

Verify email: `POST /api/v1/auth/verify-email` `{"refresh_token": "<verification token from register>"}`

Login: `POST /api/v1/auth/login` `{"email": "you@example.com", "password": "password123"}`

Your profile: `GET /api/v1/users/me` with header `Authorization: Bearer <access token>`

Refresh: `POST /api/v1/auth/refresh` `{"refresh_token": "<refresh token>"}`

Forgot password: `POST /api/v1/auth/forgot-password` `{"email": "you@example.com"}`

Reset password: `POST /api/v1/auth/reset-password` `{"token": "<reset token>", "new_password": "newpassword456"}`

Logout: `POST /api/v1/auth/logout` with header `Authorization: Bearer <access token>`